MEMORIA FINAL DEL PROYECTO

El principal sitio web para jugadores del juego de cartas coleccionables: "Pokémon TCG Pocket".

CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES WEB

AUTOR

FRANCISCO COVES GARCÍA

I.E.S. SEVERO OCHOA

Curso: 2024-2025

TABLA DE CONTENIDOS

1. INTRODUCCIÓN 1
2. INTRODUCCIÓN EN INGLÉS 1
3. OBJETIVOS 1
3.1 Objetivos fase actual 1
3.2 Objetivos fases futuras 1
4. PLANIFICACIÓN 2
4.1 Tabla de hitos 2
4.2 Diagrama de Gantt 3
5. ANÁLISIS 3
5.1 Estado del arte 3
5.2 Funcionalidades 4
6. DISEÑO 4
6.1 Requisitos técnicos 4
6.2 Arquitectura web 10
6.3 Diseño Back-end 12
6.3.1 Modelo de datos 12
6.3.2 Servicios REST 13
6.4 Diseño Front-end 16
6.4.1 Mock-ups 17
6.4.2 Guía de estilos 17
6.4.4 Capturas de la aplicación 18
7. IMPLEMENTACIÓN 21
7.1. Servidor 21
7.2. Cliente 22
8. DESPLIEGUE 24
8.1. Modelo de despliegue utilizado 24
8.2. Datos iniciales y configuración 24
8.3. Pasos para el despliegue 24
8.4. Proveedores y servicios utilizados 24
9. HERRAMIENTAS UTILIZADAS 25

1. Introducción

PokeWeb es un sitio web dirigido a la comunidad de jugadores del juego de Pokémon: ‘Pokémon Trading Card Game’.

PokeWeb ha sido creada para satisfacer las necesidades de los jugadores en diversas áreas del juego, que la aplicación oficial no contempla, mejorando así la experiencia de juego y ampliando la comunidad de jugadores.

2. Introducción en inglés

PokeWeb is a website designed for the Pokémon Trading Card Game player community.

PokeWeb was created to meet the needs of players in various areas of the game that the official app does not cover, thereby improving the gaming experience and expanding the player community.

3. Objetivos

Con la creación de PokeWeb, se consigue unir a la comunidad del juego de Pokémon y ofrecer soluciones a los problemas más frecuentes y recurrentes del juego que la aplicación oficial no contempla.

3.1. *Objetivos fase actual*

A través de PokeWeb se consigue: mostrar información sobre las cartas del juego, aplicar un sistema de filtrado sobre la información de las cartas, chatear con los jugadores y realizar intercambios de cartas.

3.2. *Objetivos fases futuras*

A fases futuras se desarrollarán las siguientes funcionalidades: elaboración y búsqueda de mazos, mayor personalización de perfil de usuario, inventario de cartas personal y automatización de intercambios.

4. Planificación

Para realizar la planificación general de la web, se dividió todo su desarrollo en 7 partes fundamentales:

1- Extracción y limpieza de datos.

2- Creación de la base de datos.

3- Diseño y estética general.

4- Implementación de sockets.

5- Desarrollo de las funcionalidades principales.

6- Pruebas y depuración.

7- Despliegue y mantenimiento.

4.1. *Tabla de hitos*

Se dispone una tabla de hitos que muestra la traza de las distintas fases del proyecto, las tareas asociadas a cada una y la estimación de días.

Extracto y limpieza de datos	Guardar imágenes en un CDN Limpieza de datos de JSON Volcado de datos en la BBDD	7 días
Base de datos	Creación de la estructura general Ampliación por funcionalidades	7 días
Diseño y estética general	Logo, tipografía y moquetado Diseño responsive	5 días
Implementación de sockets	Creación y conexión con Pusher Desarrollo de funciones real-time	7 días
Desarrollo de las funcionalidades	Sección de info. y filtro de cartas Sección de amigos y solicitudes Sección de chats Sección de intercambios Lógica control de accesos	25 días
Pruebas y depuración	Pruebas unitarias Pruebas de usuario (beta testing)	6 días
Despliegue y mantenimiento	Despliegue de Ec2 Configuración del proyecto Flujo de trabajo con GitHub	4 días

4.2. *Diagrama de Gantt*

Se muestra un diagrama de Gantt del tiempo aproximado que ha sido empleado en cada una de las fases del desarrollo del proyecto.

5. Análisis

5.1. *Estado del arte*

Existen webs que ofrecen información sobre el juego o aplicaciones móviles que simulan intercambios de cartas Pokémon. PokeWeb, junta ambas cosas y ofrece innovaciones como creación de mazos y chats entre jugadores.

PokeWeb junta a toda la comunidad del juego en un solo lugar donde puede encontrar todas las herramientas a su disposición.

5.2. *Funcionalidades*

PokeWeb ofrece una serie de funcionalidades que mejoran la experiencia de juego de los jugadores. A continuación, muestro una lista de sus características:

1. Búsqueda de cartas
2. Filtrado avanzado de cartas
3. Propuestas de intercambio
4. Gestión de amigos y usuarios
5. Sección de chats
6. Personalización de perfil de usuario
7. Gestión de usuarios y permisos
8. Búsqueda y creación de mazos

6. Diseño

6.1. *Requisitos técnicos*

Para desarrollar cada funcionalidad ha sido necesario utilizar las siguientes tecnologías y métodos para el desarrollo:

6.1.1. Búsqueda de cartas

- A través de GitHub, se ha obtenido los JSON con los datos de las cartas.
- Con Python, se descargaron las imágenes que fueron guardadas a su vez en un servicio en la nube (Cloudinary). También se utilizaron scripts para transporta la información de los JSON de las cartas dentro de la base de datos modificando las rutas para que apuntaran a las imágenes previamente guardadas en Cloudinary.

- Se han creado las vistas necesarias para realizar la vista de las cartas:
    - Sección de cartas: vista general de todas las cartas del juego .
    - Vista de la carta: vista individual y paginación de las cartas.
    - Sección de información: muestra información.
    - Sección de ataques: muestra la información de los ataques.
    - Sección de cartas relacionadas: muestra la línea evolutiva de la carta y otra versiones de la cartas seleccionada.

- Se utilizo una base de datos sql para almacenar toda la información de las cartas, creandose así 5 tablas principales:
    - Cartas: almacena la información básica.
    - Ataques: guarda el valor de los ataques.
    - Habilidades: guarda el valor de las habilidades.
    - Expansiones: guarda el valor de la expansiones en las que se encuentra cada carta.
    - Tipos: guarda el valor de los tipos de los pokemon del juego.
    - Coste de ataque: guarda el tipo de la energía que gasta al atacar.

6.1.2. Filtrado avanzado de cartas

- Se han creado las vistas necesarias para realizar el filtrado de las cartas:
    - Filtro principal: vista principal de filtros donde aparecen todos.
    - Modales de filtrado adicionales como: rareza, tipo, expansion o sobres.
    - Filtros adicionales: puntos de salud, daño de ataque, número de carta o categoría.
    - Filtros ascendentes o descendentes de rareza, número de carta o tipo.

- Para realizar el filtrado de cartas se hizo uso de PHP con Eloquent ORM que aprovecha el modelo de los objetos y sus relaciones para crear consultas sql de manera rápida y eficaz.
- Se ha utilizado React para actualizar en tiempo real los componentes y mostrar los resultados del filtrado.
- Se han creado campos y tablas adicionales en la base de datos para cubrir casos de filtrado que no estaban contemplados.

6.1.3. Propuestas de intercambio

- Se han creado las vistas necesarias para interactuar con las propuestas de intercambio:
    - Crear una propuesta: vista de creación y sección de mis propuestas.
    - Eliminar una propuesta: botón de eliminar propuesta creada.
    - Buscar una propuesta: búsqueda de propuestas por nombre o ID.
    - Ver propuesta: sección de información de la propuesta:
        - Carta solicitada: muestra las cartas solicitadas por el ofertante.
        - Detalles: muestras información de las cartas solicitadas.
        - Descripción: muestra la descripción del intercambio.
        - Información adicional: se muestra información del ofertante y fecha de publicación y ID de la propuesta.
- Creación de la lógica necesaria para reflejar el funcionamiento del juego.

- Se hace uso de 4 tablas para abordar el desarrollo de las propuestas de intercambio:
    - Propuestas: guarda las propuestas de intercambio de cada usuario.
    - Propuestas de usuario: guarda los registros de las interacciones de cada usuario con las propuestas.
    - Lineas de propuesta: guarda las cartas que se solicitan en las propuestas.
    - Items de la línea: contiene la información de la carta que se ofrece en la propuesta.

6.1.4. Gestión de amigos y usuarios

- Se han creado las siguientes vistas:
    - Ver amigos: vista de tus amigos y botones para interactuar con ellos.
    - Ver usuarios: vista de usuarios de la web.
    - Botón de solicitudes: muestra tus solicitudes de amistad.
    - Vista individual de cada usuario: se muestra información del usuario y las acciones que están permitidas con él.
        - Bloquear usuario: se bloquea a un usuario para evitar encontrarlo y recibir mensajes de su parte.
        - Botón de estado de solicitud: muestra el estado de tu solicitud de amistad con ese usuario.
        - Botón de mensaje: se abre o se crea un chat con él.
    - Búsqueda de usuarios/amigos: posibilidad de buscar amigos o usuarios.
    - Sección de amigos recientes: se ha creado una vista en la barra lateral para ver los amigos conectados/desconectados.

- En la base de datos se han creado 2 tablas que continen la lógica necesaria:
    - Amigos: contiene toda la información sobre las solicitudes de amistad entre los usuarios (aceptada, pendiente).
    - Usuarios: contiene a todos los usuarios de la web y sus datos.

6.1.5. Sección de chats

- Se han creado las siguientes vistas:
    - Sección de chats: se muestran todos los chats de tu usuario.
    - Búsqueda de chats: se puede buscar los chats por el nombre del amigo con el que tienes el chat.
    - Crear chat: seleccionar un amigo para abrir un chat con él.
    - Silenciar chat: se puede silenciar un chat para evitar ser notificado.
    - Eliminar chat: se puede eliminar un chat de tu vista de chats.
    - Sección de chat individual: se muestra el chat con el usuario y donde se intercambian mensajes entre ambos.
    - Sección de fondos: se selecciona un fondo para el chat entre ambos.
    - Sección de chats recientes: sección que aparece en la barra lateral y que muestra las ultimas interacciones.
- Se han creado las siguientes tablas en la base de datos para abordar toda la lógica de los chats y el estado de los mensajes:
    - Bloqueos: muestra la información de los bloqueos de los usuarios.
    - Chats: muestra toda la información del chat, fondo, estado del chat(silenciado o eliminado) y los usuarios que están involucrados.
    - Mensajes: muestra la información del mensaje y el estado del mismo(visto, bloqueado)

- Adjuntos de mensajes: tabla que contiene información de los mensajes con información adicional que tiene que ser almacenada aqui(stickers, propuestas de intercambio)
- Fondos: almacena la información de los fondos de los chats.

6.1.6. Personalización de perfil de usuario

- Se ha creado una vista que muestra toda la información de los usuarios:
  - Foto de perfil, nombre de usuario y día de creación del usuario.
  - Número de Amigo en el juego.
  - Modal de selección de foto de perfil.
  - Modal de introducir código de amigo.
  - Botón de cerrar/iniciar sesión.
- Se ha creado una sección de notificaciones en la barra lateral donde el usuario recibe toda la información relevante y puede consultarla.
- Se ha utilizado en la base de datos:
  - Fotos de perfil: contiene la información de las fotos de perfil.
  - Usuarios: tabla con toda la información de los usuarios.
  - Notificaciones: contiene la información de las notificaciones.

6.1.7. Gestión de usuarios y permisos

- Para gestionar todo sobre los permisos de los usuarios se ha permitido acceder a la web sin la necesidad de iniciar sesión con una cuenta.

- Se han creado las siguientes vistas:
    - Modal de iniciar sesión: vista que aparece para recordarte iniciar sesión y restringuir el uso de ciertas funcionalidades.
    - Vista principal de entradad a la web: contiene los formularios para acceder a la web, registrarte o iniciar sesión.
- Se ha permitido al usuario acceder a la web sin registrarse pero acceder solo a una funcionalidades limitadas.

6.1.8. Búsqueda y creación de mazos (*sin implementar)*

- Está planificado crear las siguientes vistas:
    - Crear mazo: vista para crear el mazo.
    - Búsqueda de mazos: sección para buscar mazos y valorarlos.
    - Mis mazos: sección donde aparecen mis mazos creado y mis mazos favoritos.
- Para la base de datos se crearían las siguientes tablas:
    - Mazos: contiene el nombre del usuario, likes y información del mazo.
    - Carta del mazo: contiene las cartas que aparecen en un mazo y la cantidad de cada una.

6.2. *Arquitectura web*

Laravel (PHP + Eloquent, MVC):

- Sirve de API REST y de capa de negocio.
- Los controladores gestionan las peticiones de React y los modelos permiten interactuar con MySQL con Eloquent.
- Eventos de Laravel para los Sockets y interacciones en tiempo real.

- Patron MVC para separar la lógica del proyecto (Modelo, Controladores y Vistas)

React (Tailwind CSS):

- Interfaz de usuario en SPA (Single Page Application).
- Uso de Tailwind CSS para los estilos
- Consume endpoints de Laravel y muestra datos.
- Conecta con Pusher(Servicio de sockets) y actualizaciones en tiempo real.

MySQL (XAMPP):

- Base de datos SQL local con las tablas: usuarios, cartas, ataques...
- Laravel usa migraciones para crear esquemas y Eloquent para consutas.

CRM: Almacenamiento en la nube que contiene las imágenes de las cartas y otros recursos utilizados en la web.

PUSHER:

- Servicio externo de sockets en tiempo real.
- Se emiten eventos desde el Backend de Laravel que son capturados por Pusher.
- React se subscribe a los canales de comunicación y recibe notificaciones sin recargar la página.

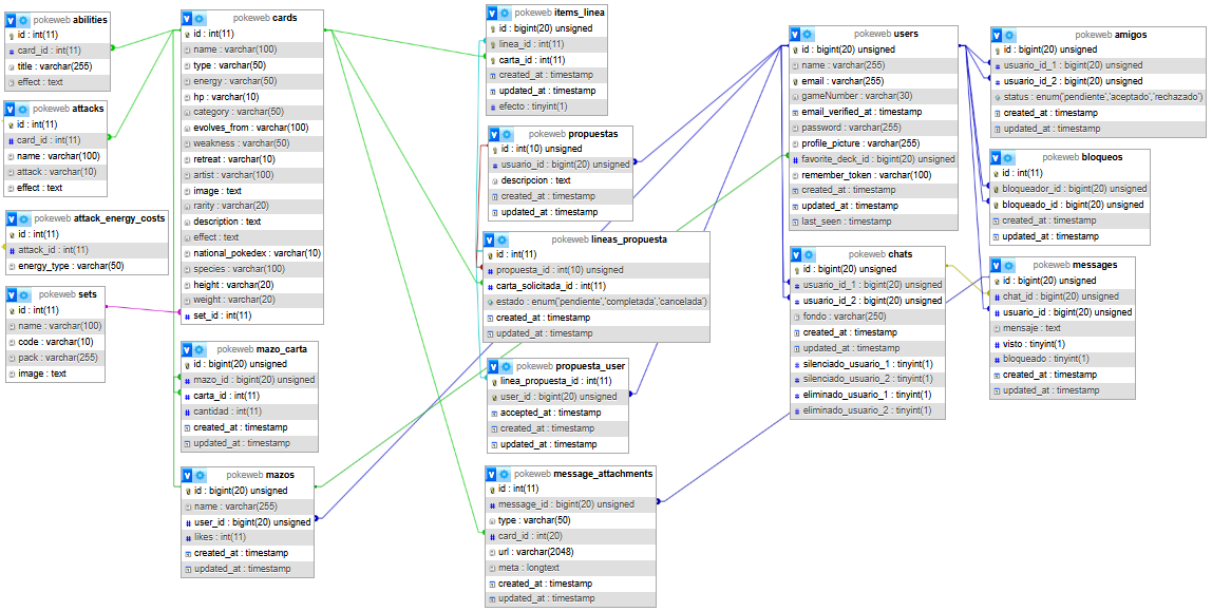
6.3. *Diseño Back-end*

Para el backend se han utilizado las siguientes herramientas:

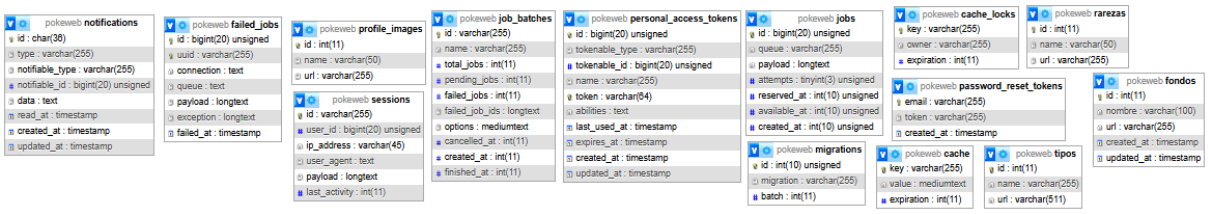
- Framework: Laravel (versión 11.42.1)
- Estructura de carpetas MVC: rutas (routes/), controladores(app/Http/Controllers), modelos (app/Models), migraciones(database/migrations), vistas (resources/js)
- ORM: Eloquent para acceder a Mysql
- Autenticación: sistema de sesiones o tokens con Laravel Sanctum.
- Configuración de Pusher: credenciales configuradas en el fichero ".env", canal y eventos configurados en el fichero "broadcasting.php".
- Conexión a la base de datos: ajustes configurados en el fichero ".env"
- Paquetes adicionales: inertia (integración SPA), laravel-echo (eventos y broadcasting), pusher (servicio de websockets), react (librería de UI), axios (cliente Http), tailwind (utilidades CSS)
- Rutas: en (routes/web.php) protegidas con middleware.

6.3.1. Modelo de datos

A continuación muestro las tablas de mi base de datos que tienen relaciones



Ahora muestro las tablas adicionales que no tiene relaciones



6.3.2. Servicios Rest

Se han utilizado los siguientes puntos de enlace (endpoints) en mi web para conectar con el Backend:

<u>Cartas</u>

GET / o GET /cartas	Obtiene el listado de cartas
GET /cartas/{carta}	Devuelve el detalle de la carta con id {carta}
GET /cartas-paginadas	Listado paginado de cartas
GET /sobres-expansiones/{expansion}	Muestra todas las cartas de la expansión {expansión}

<u>Perfil de usuario</u>

GET /perfil	Datos del perfil del usuario autenticado
GET /profile/{id}	Datos del perfil del usuario con id {id}
PATCH /profile	Actualiza los datos del usuario
DELETE /profile	Elimina la cuenta del usuario
POST /update-last-seen	Actualiza la última hora de conexión
POST /profile/disconnect	Cierra sesión del usuario
GET /profile_images	Listado de imágenes de perfil disponibles
PATCH /updateProfileImage	Cambia la imagen de perfil del usuario

Amigos

GET /amigos	Lista de amigos del usuario autenticado
GET /amigos/getAllFriends	Recupera todos los amigos (sin paginar)
GET /amigos/getFriends	Amigos filtrados
GET /usuarios/search	Busca usuarios por nombre para añadir como amistad
GET /friend-requests/{userId}	Solicitudes de amistad recibidas por el usuario con el id {userId}
POST /amigos/send	Envía solicitud de amistad
POST /amigos/cancel	Cancela solicitud de amistad pendiente
POST /amigos/accept	Acepta una solicitud de amistad
POST /amigos/reject	Rechaza una solicitud de amistad
POST /amigos/remove	Elimina de la lista de amigos
PATCH /users/{user}	Actualiza friend_id para vincular cuenta o perfil

Chats y mensajes

GET /chats	Lista de chats disponibles
GET /chats-recientes	Lista de chats recientes del usuario
GET /chats/{slug}	Datos del chat con identificador {slug}
GET /chats	Crea o accede a un chat
GET /chats/{chat}/messages	Mensajes del chat identificado por {chat}
POST /messages	Envía un mensaje nuevo a un chat
POST /chats/{chat}/visto	Marca los mensajes de un chat como leídos
POST /chats/{chat}/mute-status	Activa/Desactiva el silencio hacia un chat
GET /chats/{chat}/is-muted	Comprueba si un chat esta silenciado o no

POST /amigos/remove	Elimina de la lista de amigos
PATCH /users/{user}	Actualiza friend_id para vincular cuenta o perfil
GET /fondos	Listado de imágenes de fondo para chats
POST /chats/{chat}/fondo	Cambia el fondo de un chat
POST /chats/{chat}/eliminar	Elimina el chat de un usuario

Intercambios de cartas

GET /intercambios/nueva-propuesta	Datos necesarios para crear una propuesta de intercambio
GET /intercambio/{tab?}	Lista de propuestas y que pueden ser filtradas
GET /cartas-paginadas-propuesta	Cartas paginadas usadas al crear una propuesta
GET /propuestas-paginadas	Todas las propuestas de intercambio paginadas
GET /mis-propuestas-paginadas	Propuestas creadas por el usuario paginadas
GET /intercambios/{propuesta}	Detalles de la proupesta con id {propuesta}
POST /guardar-propuesta-intercambio	Crea y guarda una nueva propuesta de intercambio
POST /intercambios/{propuesta}/aceptar	Acepta la propuesta de intercambio
DELETE /intercambios	Elimina la propuesta {propuesta}
POST /amigos/remove	Elimina de la lista de amigos
PATCH /users/{user}	Actualiza friend_id para vincular cuenta o perfil
GET /fondos	Listado de imágenes de fondo para chats
POST /chats/{chat}/fondo	Cambia el fondo de un chat
POST /chats/{chat}/eliminar	Elimina el chat de un usuario

Notificaciones

GET /notifications	Recupera las notificaciones de un usuario
DELETE /notifications	Elimina todas las notificaciones de un usuario

Bloqueos

POST /bloquear	Bloquea a un usaurio
POST /desbloquear	Desbloquea a un usuario previamente bloqueado
POST /is-blocked/{id}	Comprueba si el usuario tiene bloqueado al usuario {id}

6.4. *Diseño Front-end*

Se ha accedido y consumido los servicios del servidor de las siguientes maneras:

- Consumo de API: Uso de Axios para llamar a los endpoints de Laravel incluyendo token de sesión en cabeceras.

- Componentes React: Creación de vistas usando hooks para manejar estados y datos.

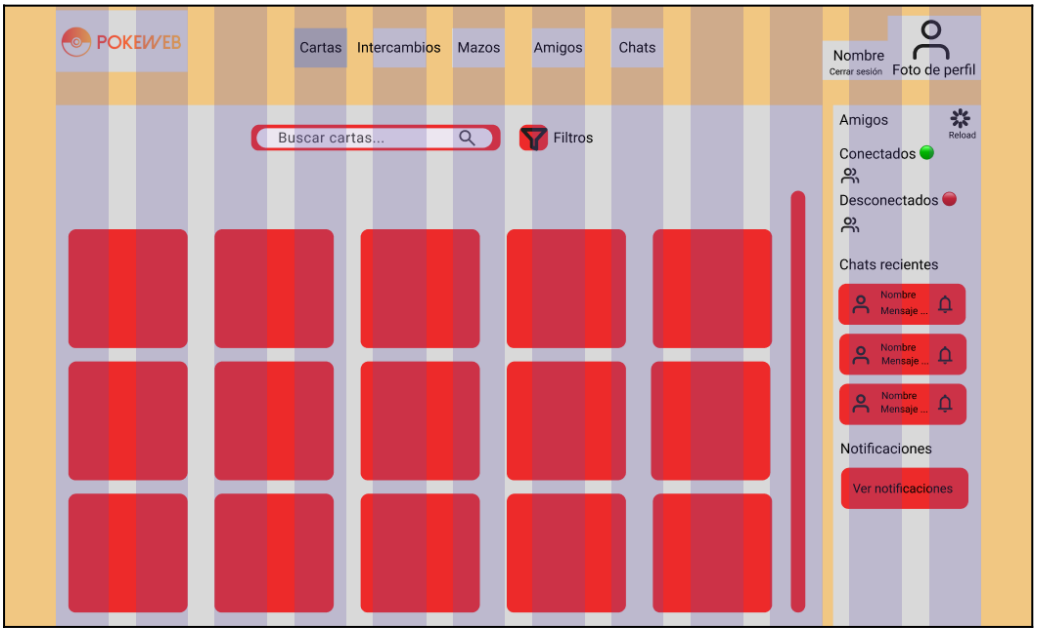
- WebSockets: Integración de Pusher a través de Laravel Echo para recibir y enviar eventos en tiempo real.

- Estilos y navegación: Tailwind CSS para diseñar de manera adaptable (responsive) y React Router y Inertia para cambiar entre las rutas protegidas y ofrecer una experiencia SPA (Single Page Application).

6.4.1. Mock-ups

Para realizar el diseño y estética general de la aplicación he hecho uso de la plataforma digital Figma para diseñar las distintas vistas de la web.

Boceto de la ventana de cartas y disposición general de la web



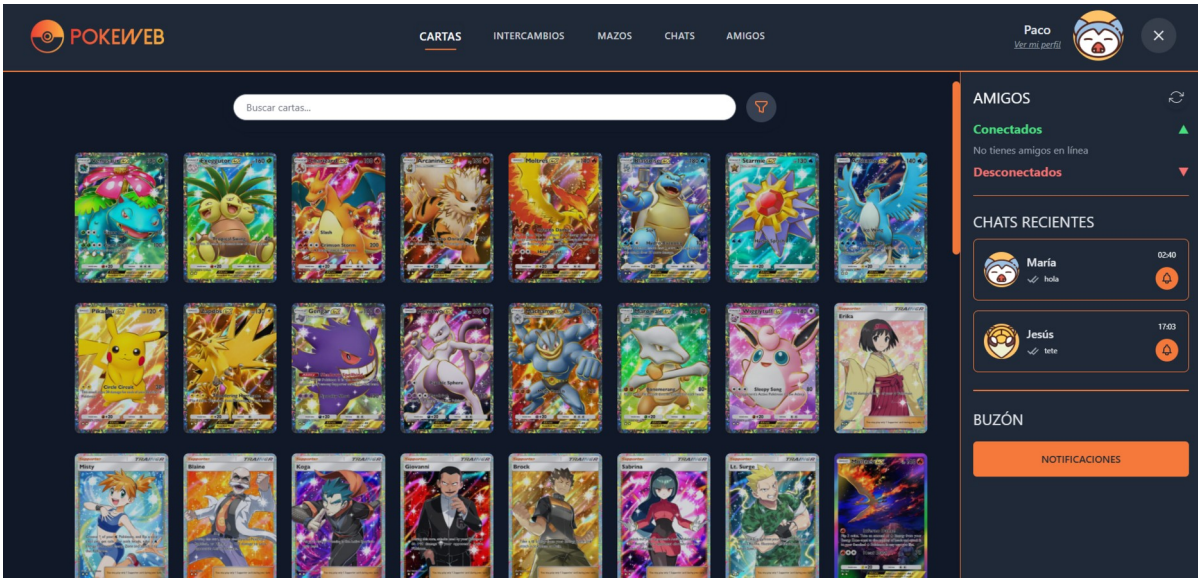
6.4.2. Guía de estilos

Muestro una guía de estilos que hice en Figma para el desarrollo de la web:

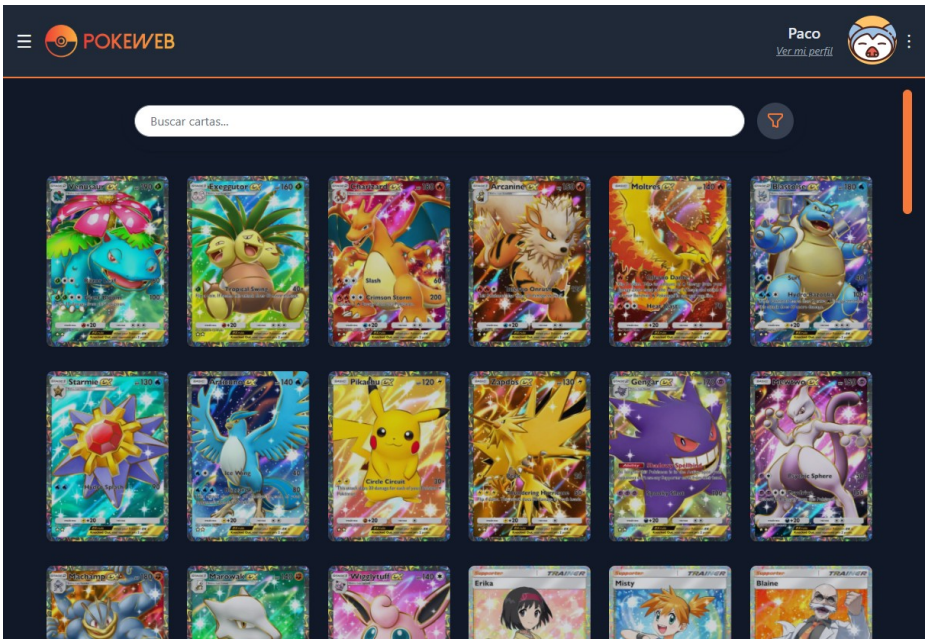
6.4.3. Capturas de la aplicación

Para crear un estilo adaptable en la web, he hecho uso de Tailwind CSS que me proporciona la comodidad necesaria para adaptar mi web y su diseño a todo tipo de pantallas.

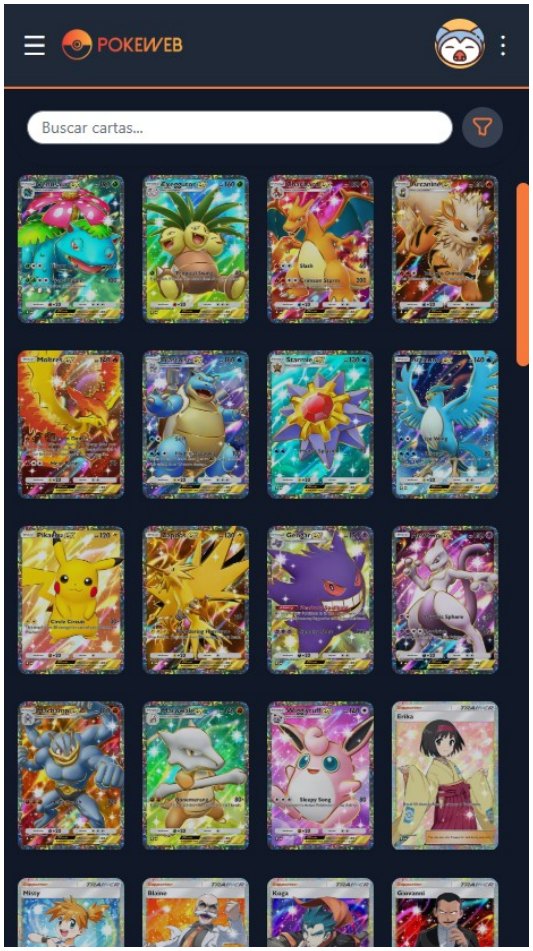
A continuación muestro la misma vista en diferentes anchos de pantalla:

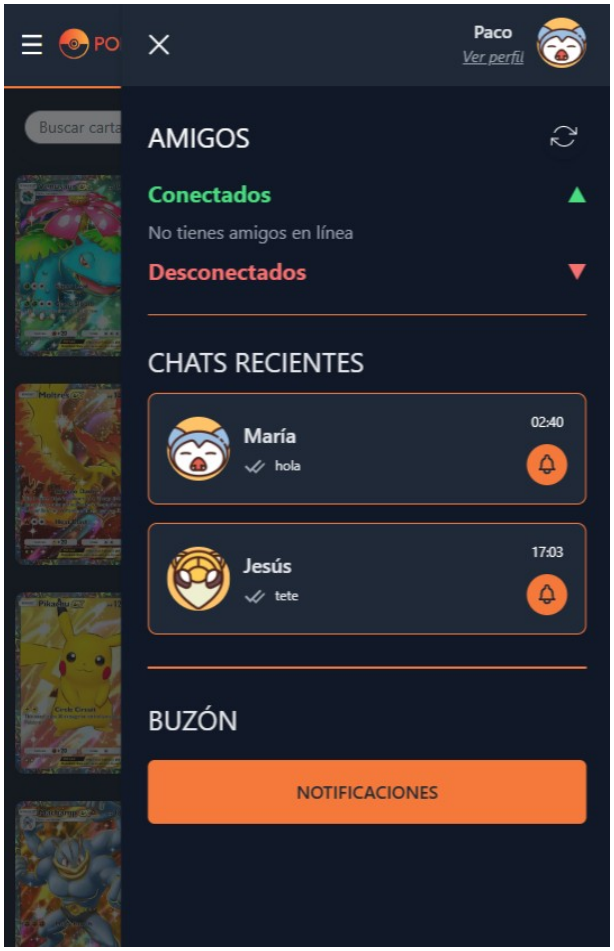


ancho de pantalla > 1300 px.



570 px. < *ancho de pantalla* < 1300 px.





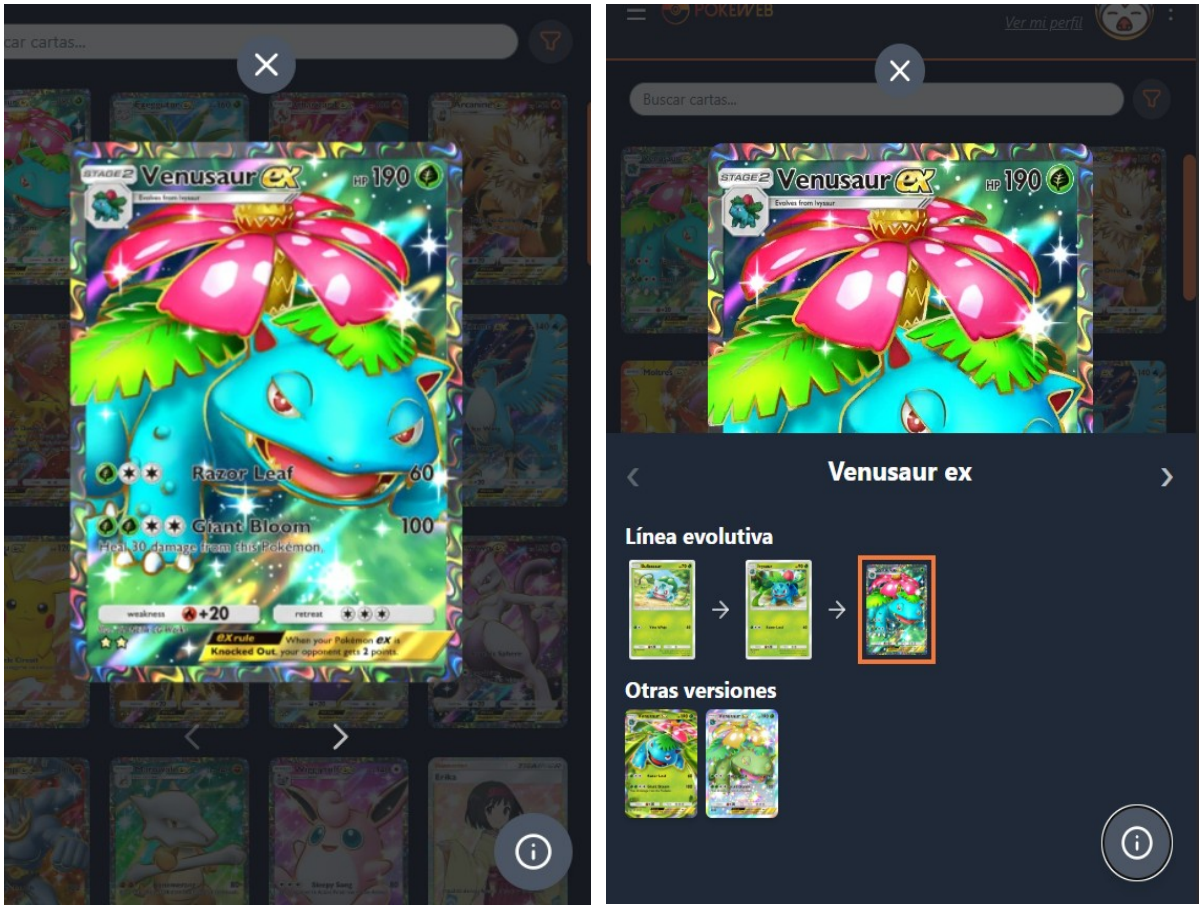
ancho de pantalla < 550 px.

Menú lateral en pantallas pequeñas

Ahora voy a mostrar una adaptación que hice en una sección de mi web, que muestra la información de las cartas, al añadir un botón flotante y dimensionar la sección para ancho de pantallas más pequeños.

Así como he realizado esta adaptación, he utilizado la misma metodología en otras secciones y partes de mi web.

He utilizado también este método en la sección de mensajería, creación de propuestas, sección de amigos, sección de chats y en los menús laterales.



Se muestra el botón flotante y la adaptación del diseño de la vista de información

7. Implementación

7.1. *Servidor*

Estas son las distintas fases del proyecto a nivel de servidor y la cronología utilizada para desarrollar PokeWeb:

1. Descargar XAMPP, composer y npm(nuget package manager) en mi ordenador.
2. Crear el proyecto: utilizar composer para crear el proyecto con laravel.
3. Instalar Vite y levantar la web con los comandos de laravel “php artisan serve” y levantar la parte de Vite con “npm run dev”.
4. Configurar el fichero “.env” añadiendo las credenciales de la base de datos, las credenciales de pusher, vite...
5. Crear la base de datos a través de XAMPP y realizar las migraciones necesarias desde laravel para llenar de datos la base de datos.
6. Crear los modelos de los objetos, los controladores y las vistas principales.
7. Definir las rutas en el fichero “routes/web.php” encargado de acceder a los controladores del Backend.
8. Configurar WebSockets en el fichero “config/broadcasting.php” con Pusher.
9. Crear los Eventos y canales de Sockets en el fichero “routes/channels.php”.
10. Autenticación y middleware: lógica de autenticación de usuarios y acceso a las rutas de la web.
11. Subir algunos recursos a la carpeta /public para utilizar en la web.
12. Crear los servicios que hacen cargo de la lógica pesada como puede ser el filtrado de cartas.
13. Conectar mi proyecto con GitHub y añadir la lógica para producción.

7.2. *Cliente*

En la parte del Front-end de mi aplicación, he utilizado React para su desarrollo.

La estructura de carpetas y ficheros que he utilizado es la siguiente:

- Public
    - C/ Images: contiene la imagen del logo de la web
- Js
    - C/ Components: se almacenan los componentes que reutilizables que se utilizan varias vistas.
    - C/ Layouts:
        - Asides: contiene los componentes menús laterales
        - Hooks: contiene ficheros de lógica que se usan en los componentes de Layouts, p.e, isDesktop que realiza la lógica para calcular el ancho de la pantalla.
        - Navbar: contiene los componentes de la barra de navegación y las secciones de la misma.
        - MainLayout.jsx: componente principal que hace uso de los demás.
    - C/ Pages
        - Amigos: carpeta que contiene la lógica de los amigos de la web
        - Auth: componente de la vista del modal de inicio de sesión
        - Cartas: carpeta que contiene la lógica de las cartas
        - Chats: carpeta que contiene la lógica de los chats
        - Intercambios: carpeta que contiene la lógica de los intercambios
        - Mazos: carpeta que contiene la lógica de los mazos

        - Perfil: carpeta que contiene la lógica del perfil del usuario
        - Dashboard: componente dashboard de la web
    - C/ Services: contiene componentes con diferente lógica de la web, p.e, crear los canales privades, públicos de sockets, componentes con contextos, funciones que llaman al backend...
    - F/ app.jsx: fichero de entrada de la web
- Routes
    - Api.php:
    - Auth.php: rutas utilizadas para el inicio de sesión, registro, acciones de contraseña...
    - Channels.php: fichero que configura los canales de broadcast de Laravel para WebSockets, p.e, canal del chat, canal público, canal privado.
    - Console.php: fichero autogenerado por laravel, utilizado para registrar mis comando por consola de Artisan.
    - Web.php: fichero principal de rutas de mi web, se definen todos los endpoints que se comunican con el servidor, rutas protegidas, rutas con inertia...

8. Despliegue

8.1. *Modelo de despliegue utilizado*

Como modo de despliegue, he hecho una instalación directa sobre una máquina de EC2 del servicio de Amazon, con Apache (XAMPP), usando el entorno LAMP sin contenedores.

He configurado el XAMPP y he traído el proyecto desde GitHub.

8.2. *Datos iniciales y configuración*

Los datos iniciales han sido las cartas pokemon, fondos de los chats, tabla de ataques, habilidades.. Por tanto, he utilizado el sistema de “seeders” que me ofrece Laravel para llenar la base de datos.

8.3. *Pasos para el despliegue*

Para realizar el despliegue, he accedido a la consola de la máquina de la instancia EC2 de Amazon y he seguido los siguientes pasos:

1- Traer el proyecto desde GitHub.

2- Configurar el fichero “.env” (MySQL, Pusher, vite).

3- Ejecutar “composer install” y “npm install && npm run build”.

4- Utilizar Laravel para realizar la migración de las tablas en mi base de datos de la máquina EC2.

5- Ajustar algunos permisos de ficheros.

6- Apuntar el Virtual Host al public/ de Laravel.

8.4. *Proveedores y servicios utilizados*

Para hospedar mi proyecto web he hecho uso de el servicio EC2 de Amazon, que permite su uso de forma gratuita mientras no superes una cierta cantidad de horas de despliegue.

Para guardar mis imágenes, he utilizado un CRM (servicio de almacenamiento en la nube) llamado Cloudinary. Para trabajar con Sockets he utilizado un servicio en la nube gratuito llamado Pusher.

9. Herramientas utilizadas

Para finalizar voy a explicar las principales herramientas utilizadas para el desarrollo general del proyecto:

1- XAMPP (base de datos, MySQL y aplicación Laravel)

2- Laravel (framework utilizado con un patrón MVC)

3- Node.jsx / npm (gestionar dependencias del frontend “React, Tailwind, Vite”)

4- Composer (instalar librerías de Laravel y Eloquent)

5- Vite (compilador de assets de frontend)

6- Visual Studio Code (editor de texto para trabajar)

7- GitHub (versionado de la web en la nube y local)

8- PHP (lenguaje utilizado para el backend de la web)

9- JavaScript (lenguaje para la parte del front de la web)

10- React (biblioteca utilizada para componetizar la aplicación)

11- Tailwind CSS (framework de utilidades CSS)

12- Cloudinary (servicio de almacenamiento en la nube gratuito)

13- Pusher (servicio de sockets en la nube gratuito)

14- Eloquent (ORM que incluye Laravel para trabajar con objetos y interactuar con la base de datos fácilmente)

15- Librerías de React (utilizadas para implementar funcionalidades ya desarrolladas de manera rápida)